

SOFTWARE REQUIREMENTS SPECIFICATION

Intern Management System (IMS)

Version 1.0 | Confidential

Prepared for Development Team

June 2026

Document Information

Document Title	Software Requirements Specification – Intern Management System
Version	1.0
Status	Final – Ready for Development
Date	June 2026
Audience	Development Team (MERN Stack Engineers)
Platform	Web Application (Desktop & Tablet Browsers)
Tech Stack	MongoDB · Express.js · React.js · Node.js
Confidentiality	Internal Use Only

Table of Contents

1. Introduction & Purpose
2. System Overview
3. Roles & Access Control
4. Marking & Scoring System
5. Functional Requirements – Admin
6. Functional Requirements – Teacher
7. Functional Requirements – Student
8. Automated Lecture Pipeline
9. Attendance Module
10. Quiz Module
11. Assignment & Code Review Module
12. Curriculum Management
13. Recruiter Portfolio & Shareable Link
14. Recruiter Metrics – Extended Analytics
15. Notification System
16. Admin Override & Audit Trail
17. Database Schema
18. API Endpoints
19. Non-Functional Requirements
20. Constraints & Assumptions
21. Glossary

1 Introduction & Purpose

1.1 Purpose

This Software Requirements Specification (SRS) defines every functional and non-functional requirement for the Intern Management System (IMS). The document is the single source of truth for the development team. All features, rules, data flows, scoring formulas, edge cases, and admin overrides are fully specified. There are no open-ended decisions left to the developer's discretion unless explicitly marked as implementation-detail.

1.2 Project Goal

The IMS automates the full lifecycle of an internship cohort – from curriculum upload through lecture delivery, attendance, quizzes, assignments, and scoring – and produces a rich, data-backed portfolio for each intern that a recruitment agency can evaluate via a secure shareable web link. Human intervention is reduced to the initial curriculum setup and periodic lecture delivery. All scoring, tracking, and reporting is system-driven.

1.3 Scope

- Web-only application (no mobile app); responsive layout for desktop and tablet browsers.
- MERN stack: MongoDB, Express.js, React.js, Node.js.
- Four distinct user roles: Admin, Teacher, Student, Recruiter (read-only via link).
- Integrates with Google Meet (scheduled links), Kahoot/Google Forms (quiz scores imported via CSV/manual upload), and an existing third-party code-review service (returns score + feedback).
- Out of scope: video hosting, in-app video streaming, payment processing, mobile native apps.

1.4 Definitions

Term	Definition
Batch	A cohort of students enrolled in one curriculum instance.
Topic	The atomic unit of the curriculum (curriculum is divided by topics, not weeks).
Lecture	A scheduled online session (Google Meet) tied to one or more topics.
Attendance CSV	A file exported from the third-party attendance extension listing students who attended.
Half-Attendance	Attending only one of the two halves of a lecture session.
Base Score	The starting mark every student receives upon enrollment: 30.
Mark Cap	The maximum total marks a student can hold: 100.
Negative Marks	Marks can fall below 30 and below 0 if deductions exceed additions.

Code Review	Automated analysis of a student's submitted GitHub repository; returns a score (0–10) and feedback text.
Recruiter Link	A publicly accessible, read-only, batch-level URL listing all student portfolios.

2 System Overview

2.1 High-Level Architecture

The IMS is a single-page React application backed by a Node.js/Express REST API and a MongoDB database. All business logic (scoring, mark calculation, portfolio generation) resides in the backend. The frontend consumes the REST API and renders dashboards per role.

2.2 Core Automated Pipeline

The system enforces the following pipeline for every lecture event. Each step is triggered automatically; the teacher performs only manual actions explicitly listed.

Step	Actor	System Action
1. Schedule Lecture	Teacher	Teacher creates lecture record: selects topic(s), sets date/time, pastes Google Meet link, sets assignment deadline.
2. Lecture Notification	System	System sends email/in-app notification to all enrolled students with lecture details 24 hrs and 1 hr before start.
3. Lecture Held	Teacher	Teacher conducts session on Google Meet externally.
4. Attendance Upload	Teacher	Teacher exports CSV from third-party attendance extension and uploads it to IMS attendance module.
5. Attendance Processing	System	System parses CSV, matches student IDs/emails, applies mark rules, auto-recalculates totals.
6. Quiz Upload	Teacher	Teacher uploads CSV of student quiz scores (Kahoot export or Google Forms export).
7. Quiz Processing	System	System validates scores (0–5), applies additions/deductions, flags anomalies for admin review.
8. Assignment Issued	System	Assignment record (GitHub repo URL + deadline) is auto-published to student dashboard when lecture is marked complete.
9. Student Submission	Student	Student submits GitHub repo URL within deadline via dashboard.
10. Code Review	System	System calls existing code review service; receives score (0–10) + feedback; stores result.
11. Score Propagation	System	All three scores (attendance + quiz + assignment) are summed and added to running total; portfolio updated.
12. Portfolio Refresh	System	Recruiter-facing batch link is updated with new data in real time.

3 Roles & Access Control

3.1 Role Summary

Role	Login Method	Count	Auth Level
Admin	Email + Password	4 accounts (equal permissions)	Full system access including overrides and audit log
Teacher	Email + Password	Unlimited accounts	Create/manage curriculum, lectures, upload attendance & quiz CSVs, view student data
Student	Email + Password	Unlimited accounts	View own dashboard, submit assignments, view own portfolio
Recruiter	No login	N/A	Read-only access via batch-specific shareable URL; no authentication required

3.2 Admin

- 4 admin accounts exist in the system. All 4 have identical permissions.
- Admin accounts are created at system setup and cannot be created via the UI (seeded via backend script).
- Admin can: create/edit/delete teacher accounts, create/edit/delete student accounts, create batches, assign students to batches, assign teachers to batches.
- Admin can view and override any student's marks with a mandatory reason field (all overrides are logged).
- Admin can correct any teacher-uploaded CSV and trigger system recalculation.
- Admin has access to a full audit trail of all mark changes.
- Admin can generate and revoke recruiter batch links.
- Admin can configure batch-level settings (see Section 4.8).

3.3 Teacher

- Teacher accounts are created by Admin.
- A teacher is assigned to one or more batches.
- Teacher can see only the batches and students assigned to them.
- Teacher has a profile page: name, photo (upload), designation, bio, contact email.
- Teacher actions: create topics, create lectures, upload attendance CSV, upload quiz CSV, mark lecture as complete (triggers assignment issuance).
- Teacher cannot modify student marks directly; all mark changes go through the CSV upload or admin override.

3.4 Student

- Student accounts are created by Admin. Each student is assigned to exactly one batch.
- Student can log in and view: their current total marks, per-lecture breakdown, per-quiz scores, per-assignment scores, and their full portfolio.
- Student can submit assignment (GitHub repo URL) before the deadline. Submission after deadline is accepted by the system but auto-scored as 0.
- Student cannot edit or delete their submission once submitted.
- Student can view code review feedback on their assignments.
- Student profile: name, photo, contact email, batch, enrollment date.

3.5 Recruiter (Public Link)

- No authentication required. Access is via a unique, batch-specific URL.
- URL is generated by Admin and can be revoked and regenerated by Admin.
- The link displays a list of all students in the batch with portfolio cards.
- Each student card expands to show all metrics defined in Section 14.
- No personally sensitive data (phone numbers, personal email) is shown to the recruiter.

4 Marking & Scoring System

4.1 Starting Marks

Every student begins with a base score of 30 marks upon enrollment in a batch. This is stored as the initial value in the student's mark ledger.

Rule: Starting marks = 30. This value is set at enrollment and cannot be changed by a teacher. Only an Admin can retroactively adjust starting marks with a logged reason.

4.2 Mark Cap

A student's total marks are capped at a maximum of 100. The system must enforce this cap at the point of every mark addition. If adding marks would push the total above 100, the system records only enough marks to reach 100 and stores the overflow as 'excess marks' (not counted toward total but visible in the student's ledger for transparency).

Rule: Max total = 100. Excess marks are logged but not added to the total. Marks can fall to any negative value; there is no floor.

4.3 Attendance Marks

Each lecture is split into two halves (Half 1 and Half 2). The attendance CSV uploaded by the teacher must contain a column indicating which half(ves) each student attended.

Attendance Scenario	Marks Applied	Notes
Both halves attended	+2.5 marks	Full attendance for the lecture.
Only one half attended	+1.0 mark (not +1.25)	Half-attendance gives flat 1.0, not 1.25. Missing half = -1.5 is not separately deducted; total is simply 1.0.
No halves attended	-5.0 marks	Full deduction for missing the entire lecture.

No Loophole: The attendance CSV is the sole source of truth. If a student is not listed in the CSV for a given lecture, the system treats them as absent (-5.0). The teacher cannot manually mark individual students; only CSV upload is accepted.

4.4 Quiz Marks

Each quiz is scored out of 5. The teacher uploads a CSV of student quiz scores after each quiz session.

Scenario	Marks Applied	Notes
Student participated, scored S ($0 \leq S \leq 5$)	+S marks	Score is added directly to total.
Student did not participate (absent from quiz)	-2.5 marks	Deduction applied if student email/ID is absent from quiz CSV.
Student scored 0 (participated, scored zero)	+0 marks	No deduction; 0 score means no addition but no penalty.

Validation: If the uploaded CSV contains a score > 5 for any student, the system rejects the entire CSV upload and displays an error listing the anomalous rows. The teacher must correct and re-upload.

4.5 Assignment Marks

Each lecture may have exactly one associated assignment. Assignments are auto-scored by the existing code review system, which returns a numeric score between 0 and 10 (inclusive) and a text feedback block.

Scenario	Marks Applied	Notes
Submitted on time, code review returns score S	+S marks (0–10)	Score added to total. Feedback stored and visible to student.
Submitted after deadline	+0 marks	Submission is accepted and code review still runs (for feedback), but score is recorded as 0.
Not submitted	+0 marks	No deduction for missing assignment; student simply loses the opportunity to gain marks.
Code review service returns error	Mark held as Pending	Admin is notified. Admin can manually enter score after resolving. Logged in audit trail.

Deadline Rule: Deadline is set by the teacher at lecture creation time. The system timestamps the student's submission at server receipt time (not client submit time). Any submission with server timestamp $>$ deadline is auto-scored 0, regardless of reason.

4.6 Mark Calculation Formula

The total marks for a student at any point in time is computed as:

$$\begin{aligned}
 \text{Total Marks} &= 30 \quad (\text{base}) \\
 &+ \sum \text{Attendance Marks (all lectures)} \\
 &+ \sum \text{Quiz Marks (all quizzes)} \\
 &+ \sum \text{Assignment Scores (all assignments)}
 \end{aligned}$$

capped at 100 (no floor)

4.7 Recalculation on CSV Correction

- If a teacher re-uploads an attendance or quiz CSV for a lecture that already has processed marks, the system detects the duplicate and triggers a full recalculation.
- The system computes the delta between the old marks and the new marks for each affected student and applies the delta to their running total.
- The recalculation is atomic: either all students are updated or none are (rollback on failure).
- Every recalculation event is logged in the audit trail with: timestamp, teacher ID, lecture ID, old marks per student, new marks per student.
- Admin is notified of any recalculation event.

4.8 Admin-Configurable Mark Parameters

Admin can adjust the following parameters at batch level. Changes take effect from the next event (not retroactively unless admin explicitly triggers a full recalculation).

Parameter	Default Value	Admin-Editable	Notes
Base starting marks	30	Yes	Changing this triggers full recalculation for all students in batch.
Full attendance marks	+2.5	Yes	
Half attendance marks	+1.0	Yes	
Missed lecture deduction	-5.0	Yes	
Quiz max score	5	Yes	CSV validation threshold updated accordingly.
Missed quiz deduction	-2.5	Yes	
Assignment max score	10	No	Fixed; determined by code review system output range.
Late assignment score	0	No	Fixed by business rule.
Mark cap	100	Yes	Lowering cap triggers immediate re-capping for all students.

5 Functional Requirements – Admin

5.1 Account Management

- FR-A-01: Admin can create teacher accounts (name, email, password, photo, bio, designation).
- FR-A-02: Admin can deactivate/reactivate teacher accounts. Deactivated teachers cannot log in.
- FR-A-03: Admin can create student accounts (name, email, password, batch assignment, enrollment date).
- FR-A-04: Admin can deactivate/reactivate student accounts.
- FR-A-05: Admin can bulk-create student accounts by uploading a CSV (columns: name, email, batch).
- FR-A-06: Admin cannot delete accounts (only deactivate) to preserve data integrity.

5.2 Batch Management

- FR-A-07: Admin can create a new batch (name, start date, end date, assigned teacher(s), assigned students).
- FR-A-08: Admin can assign multiple teachers to a single batch.
- FR-A-09: Admin can move a student from one batch to another. Marks from the old batch are frozen; new batch starts at 30.
- FR-A-10: Admin can close a batch. Closed batches are read-only; no new lectures, quizzes, or assignments can be added.
- FR-A-11: Admin can generate a recruiter link for a batch. The link is a UUID-based URL (e.g., /recruiter/{uuid}).
- FR-A-12: Admin can revoke and regenerate a recruiter link. Old link becomes immediately invalid (returns 404).

5.3 Mark Override

- FR-A-13: Admin can add or subtract an arbitrary number of marks to any student's total with a mandatory reason field (minimum 20 characters).
- FR-A-14: Admin can override a specific lecture's attendance mark, quiz mark, or assignment mark for any student with a mandatory reason.
- FR-A-15: All overrides are logged to the audit trail (see Section 16).
- FR-A-16: Admin can trigger a manual full recalculation for any student or any batch at any time.
- FR-A-17: Admin can set the mark parameters listed in Section 4.8 per batch.

5.4 System Configuration

- FR-A-18: Admin can configure notification settings (enable/disable email notifications, set email sender details).
- FR-A-19: Admin can view and export the full audit trail as a CSV.
- FR-A-20: Admin can view a dashboard showing: total batches, total students, total teachers, top and bottom scoring students across all batches.

6 Functional Requirements – Teacher

6.1 Teacher Profile

- FR-T-01: Teacher can view and edit their own profile: name, photo, designation, bio, contact email.
- FR-T-02: Teacher profile is publicly visible on the recruiter batch page (name and designation only).

6.2 Curriculum Management

- FR-T-03: Teacher can create, edit, and delete topics within their assigned batch's curriculum.
- FR-T-04: Each topic has: title, description (rich text), learning objectives (list), estimated duration (hours), and order index.
- FR-T-05: Teacher can upload supporting notes per topic (PDF or text). Max file size: 10 MB per file. Accepted formats: PDF, .md, .docx.
- FR-T-06: Teacher can reorder topics by dragging (order index updates accordingly).
- FR-T-07: Deleting a topic is only allowed if no lecture is linked to it. If a lecture is linked, the system blocks deletion and shows an error.

6.3 Lecture Scheduling

- FR-T-08: Teacher can create a lecture record with: topic(s) covered (multi-select), date, start time, end time, Google Meet URL, half-1 end time (used as the boundary for split attendance).
- FR-T-09: Lecture record has a status field: Scheduled → In Progress → Completed.
- FR-T-10: Teacher manually marks a lecture as Completed. This action triggers assignment issuance to the student dashboard.
- FR-T-11: Teacher cannot mark a lecture as Completed before its scheduled end time (system enforces this with a timestamp check).
- FR-T-12: Teacher can edit lecture details (time, Meet URL) only while status is Scheduled.
- FR-T-13: Teacher can cancel a lecture (status = Cancelled). Cancelled lectures apply no marks to any student (neither positive nor negative).
- FR-T-14: Teacher sets the assignment deadline at lecture creation time. Deadline must be at least 1 hour after lecture end time.

6.4 Attendance Upload

- FR-T-15: Teacher uploads a CSV file exported from the third-party attendance extension. Required columns: student_email, half_attended (values: '1', '2', 'both', 'none').
- FR-T-16: System validates the CSV: checks required columns exist, emails match enrolled students, half_attended value is one of the allowed values.
- FR-T-17: Validation errors are shown per row with line numbers. Teacher must correct and re-upload; partial processing is not allowed.
- FR-T-18: Students present in the batch but absent from the CSV are automatically recorded as absent (−5.0).

- FR-T-19: Teacher can re-upload a corrected CSV for the same lecture. System triggers recalculation (see Section 4.7).
- FR-T-20: Teacher cannot upload attendance for a lecture that has status Cancelled or Scheduled (must be In Progress or Completed).

6.5 Quiz Score Upload

- FR-T-21: Teacher uploads a CSV of quiz scores. Required columns: student_email, score (numeric, 0–5).
- FR-T-22: CSV validation: scores must be numeric and within 0–5 inclusive. Any out-of-range score causes full rejection of the upload.
- FR-T-23: Students in the batch but absent from the CSV are recorded as quiz-absent (–2.5).
- FR-T-24: Teacher can re-upload a corrected quiz CSV. Recalculation is triggered.
- FR-T-25: Teacher selects which lecture this quiz is associated with at upload time.

6.6 Teacher Dashboard

- FR-T-26: Teacher dashboard shows: list of their batches, upcoming lectures (next 7 days), pending attendance uploads (lectures marked Completed but no CSV uploaded yet), pending quiz uploads.
- FR-T-27: Teacher can view a per-student mark breakdown for their batch.
- FR-T-28: Teacher can view a per-lecture summary: number of students who attended, average quiz score, average assignment score.

7 Functional Requirements – Student

7.1 Student Dashboard

- FR-S-01: Student sees their current total marks prominently on the dashboard.
- FR-S-02: Student sees a marks history table: one row per event (lecture/quiz/assignment), showing date, event type, marks applied, running total after event.
- FR-S-03: Student sees a progress bar from 30 to 100, filled to their current marks.
- FR-S-04: Student sees their rank within their batch (e.g., 3rd out of 20) updated in real time.
- FR-S-05: Student sees upcoming lectures (date, time, Google Meet link, topic).
- FR-S-06: Student sees pending assignments with deadlines and a countdown timer.

7.2 Assignment Submission

- FR-S-07: Student submits a GitHub repository URL for an assigned assignment via a text input field on the assignment detail page.
- FR-S-08: System validates that the URL matches the pattern `https://github.com/{user}/{repo}` before accepting submission.
- FR-S-09: After submission, system immediately calls the code review service in the background. Student sees status: 'Submitted – Review Pending'.
- FR-S-10: Once code review completes, student sees: score (0–10), feedback text, and whether submission was on time or late.
- FR-S-11: Student cannot edit or delete their submission after submission.
- FR-S-12: If the deadline has passed and the student has not submitted, the assignment shows as 'Missed – 0 marks'. No submission input is shown.

7.3 Student Portfolio View

- FR-S-13: Student can view their own portfolio (same view as the recruiter sees for that student).
- FR-S-14: Portfolio includes all metrics listed in Section 14.
- FR-S-15: Student can download their portfolio as a PDF (system generates a PDF server-side).

8 Automated Lecture Pipeline

This section defines every automated trigger and state transition in the lecture lifecycle. No step requires human action beyond what is explicitly stated.

Trigger	Automated System Action	Data Stored
Teacher creates lecture	Lecture record created with status = Scheduled. Notification job scheduled for T-24h and T-1h.	lecture_id, topic_ids, date, start_time, end_time, half1_end_time, meet_url, deadline, status=Scheduled
T-24h before lecture start	Email + in-app notification sent to all enrolled students and the assigned teacher.	notification_log entry
T-1h before lecture start	Email + in-app reminder sent to all students.	notification_log entry
Teacher marks lecture In Progress	Lecture status updated to In Progress. Attendance CSV upload becomes available.	status=InProgress, actual_start_time
Teacher marks lecture Completed	Lecture status = Completed. Assignment record published to student dashboard with deadline. Attendance & quiz uploads enabled.	status=Completed, assignment published
Teacher uploads attendance CSV	CSV parsed, validated. Marks applied per rules in Section 4.3. Mark ledger updated for each student. Recalculation runs.	attendance_record per student, mark_ledger entries
Teacher uploads quiz CSV	CSV parsed, validated. Quiz marks applied per rules in Section 4.4. Mark ledger updated.	quiz_record per student, mark_ledger entries
Student submits assignment before deadline	GitHub URL stored. Code review API called asynchronously. Status = Review Pending.	assignment_submission: url, submitted_at, status=pending
Student submits assignment after deadline	GitHub URL stored. Code review API called asynchronously. Score will be stored but overridden to 0 in mark ledger. Status = Late.	assignment_submission: url, submitted_at, late=true
Code review API returns result	Score and feedback stored. If late, mark_ledger entry = 0. If on time, mark_ledger entry = returned score. Portfolio refreshed.	assignment_submission updated, mark_ledger entry, portfolio_snapshot
Code review API returns error	Submission status = Review Error. Admin notified via in-app alert and email. Mark held as Pending until admin resolves.	admin_alert, mark status=pending
Admin triggers manual recalculation	Full recompute of student total from all ledger entries in chronological order. Cap enforced.	recalc_log entry

9 Attendance Module

9.1 CSV Format Specification

The attendance CSV exported from the third-party extension must conform to the following format before upload. The system does not transform or guess column names; it enforces exact header names.

Column Name	Data Type	Required	Allowed Values
student_email	string	Yes	Must match a student email in the batch.
half_attended	string	Yes	Exactly one of: 'both', '1', '2', 'none'. Case-insensitive.

9.2 Processing Logic

1. System receives CSV file (max size: 5 MB).
2. System checks: file is CSV, headers are present and correct.
3. System iterates each row. For each row: validate email exists in batch, validate half_attended value.
4. If any validation error: reject entire CSV, return error list with row numbers and error descriptions.
5. If all rows valid: apply marks per table in Section 4.3 to each student.
6. For each student in the batch NOT present in the CSV: record half_attended = 'none', apply -5.0.
7. Write all mark_ledger entries with: student_id, lecture_id, event_type='attendance', marks_applied, timestamp.
8. Recompute running total for each affected student, enforcing cap.
9. Return success response with per-student mark summary.

9.3 Re-upload / Correction

- If a CSV already exists for a lecture, re-upload triggers recalculation.
- System fetches old mark_ledger entries for this lecture with event_type='attendance'.
- System reverses old entries (creates reversal ledger entries with opposite sign).
- System applies new entries from the corrected CSV.
- Audit trail entry created: { lecture_id, teacher_id, old_csv_id, new_csv_id, delta_per_student[], timestamp }.

10 Quiz Module

10.1 Quiz CSV Format

Column Name	Data Type	Required	Allowed Values / Notes
student_email	string	Yes	Must match a student email in the batch.
score	number	Yes	Numeric. $0 \leq \text{score} \leq 5$. Decimal values allowed (e.g., 3.5). If > 5 , entire CSV is rejected.

10.2 Processing Logic

10. System receives CSV, validates format (same two-phase approach as attendance).
11. For each row: validate email in batch, validate score range.
12. If any score > 5 : reject entire CSV with row-level error.
13. For each valid row: write mark_ledger entry with event_type='quiz', marks_applied = score value.
14. For each student in batch NOT present in CSV: write mark_ledger entry, marks_applied = -2.5 (missed quiz).
15. Recompute running total. Return per-student summary.

10.3 Quiz–Lecture Association

- Each quiz CSV upload is associated with exactly one lecture at upload time (teacher selects from dropdown of Completed lectures).
- A lecture can have at most one quiz CSV. Re-upload replaces the existing one and triggers recalculation.
- A lecture can have no quiz (teacher may skip quiz upload). Students in that lecture are not penalised for the absence of a quiz.

11 Assignment & Code Review Module

11.1 Assignment Record

Field	Type	Set By	Notes
assignment_id	UUID	System	Auto-generated.
lecture_id	ref	System	Linked to the parent lecture.
topic_ids	ref[]	Teacher	Topics covered in this assignment (carried from lecture).
title	string	Teacher	Short title of the assignment (set at lecture creation).
description	rich text	Teacher	Assignment brief / requirements.
github_repo_seed	string	Teacher	Optional: a starter repo URL that students should fork.
deadline	datetime	Teacher	Set at lecture creation. Minimum: lecture end_time + 1 hour.
status	enum	System	Values: Unpublished, Published, Closed.

11.2 Student Submission Record

Field	Type	Notes
submission_id	UUID	Auto-generated.
assignment_id	ref	
student_id	ref	
github_url	string	Validated pattern: https://github.com/{user}/{repo}
submitted_at	datetime	Server-side timestamp. Used to determine if late.
is_late	boolean	True if submitted_at > deadline.
review_status	enum	Values: Pending, Completed, Error.
review_score	number	0–10. Returned by code review service.
review_feedback	text	Full feedback text returned by code review service.
mark_awarded	number	Equals review_score if on time; 0 if late or error-pending.

11.3 Code Review Integration

- The system calls the existing code review API endpoint with the student's GitHub URL.
- The API call is made asynchronously via a background job queue (e.g., Bull/Redis). Student dashboard shows 'Review Pending' until complete.

- On success: review_status = Completed, review_score and review_feedback stored. mark_awarded = review_score (or 0 if late).
- On error: review_status = Error, admin notified, mark_awarded = Pending. Admin must manually enter mark or retry review.
- Retry: Admin can trigger a re-review for any submission in Error status. Retry calls the same code review API again.
- The code review service is treated as a black box: IMS sends GitHub URL, receives { score: number, feedback: string }. No other assumption about the service internals is made in this document.

11.4 Assignment Lifecycle

Status	Condition	Student Can Submit?
Unpublished	Lecture not yet marked Completed.	No.
Published	Lecture marked Completed, deadline not yet passed.	Yes.
Closed	Deadline passed.	No. Submissions after deadline are not shown.

12 Curriculum Management

12.1 Curriculum Structure

The curriculum is a flat, ordered list of topics (not hierarchical). Each topic is independent and can be linked to one or more lectures.

Field	Type	Notes
topic_id	UUID	Auto-generated.
batch_id	ref	Curriculum belongs to a batch.
title	string	Max 120 characters.
description	rich text	HTML-formatted content. Supports headings, bold, lists, code blocks.
learning_objectives	string[]	Array of objective strings. Min 1, max 10.
estimated_hours	number	Positive decimal. Used in portfolio for total curriculum hours.
order_index	integer	Determines display order. Unique within batch.
notes_files	string[]	Array of file storage URLs. Max 5 files per topic.
created_at	datetime	Auto-set.
updated_at	datetime	Auto-updated on any edit.

12.2 Notes Upload Rules

- Accepted file types: PDF, .md, .docx.
- Max file size per file: 10 MB.
- Max 5 files per topic.
- Files are stored in the backend file storage (S3-compatible or local, developer decision on infra). The path is stored in notes_files array.
- Students can view/download notes files from the topic view on their dashboard.

12.3 Curriculum Visibility

- Students can view the full curriculum (all topics, descriptions, objectives, notes) for their enrolled batch.
- Students can see which topics are covered by completed lectures vs upcoming lectures.
- Recruiter link shows curriculum completion percentage (topics with at least one completed lecture / total topics).

13 Recruiter Portfolio & Shareable Link

13.1 Link Specification

- URL format: `https://{domain}/recruiter/{batch_uuid}`
- The `batch_uuid` is a randomly generated UUIDv4 (not the internal batch ID).
- The link requires no login. Any person with the URL can access the page.
- If the link is revoked by Admin, the URL returns HTTP 404 with a message: 'This link is no longer active.'
- The link is regenerable by Admin. Regeneration creates a new UUID; the old UUID returns 404.

13.2 Batch Overview Page (Recruiter View)

When a recruiter opens the link, they see:

- Batch name, program description, start and end dates.
- Number of students enrolled.
- Curriculum overview (list of topics, total curriculum hours).
- Teacher name(s) and designation (no contact details).
- A searchable, sortable table of all students with: name, current total marks, rank, attendance rate (%), on-time submission rate (%), average assignment score, consistency score (defined in Section 14).
- Clicking a student opens their individual portfolio card (all metrics from Section 14).

13.3 Individual Student Portfolio Card

- Student name and profile photo.
- Current total marks with a visual gauge (30 to 100 scale).
- Rank within batch.
- All metrics from Section 14 displayed with labels and values.
- Assignment history: per-assignment score, on-time/late status, topic covered.
- Code review feedback excerpts (summary only; full feedback is not shown to recruiter).
- Curriculum progress: which topics completed.
- No personally identifiable contact information beyond name and photo.

14 Recruiter Metrics – Extended Analytics

All metrics below are computed by the system automatically from event data. No manual input is required from teacher or admin to generate these metrics. Each metric is defined with its exact formula.

14.1 Academic Performance Metrics

Metric	Formula / Definition	Purpose for Recruiter
Total Score	Sum of all mark_ledger entries + 30 base, capped at 100.	Overall performance indicator.
Batch Rank	Position when all students in batch sorted descending by Total Score.	Relative standing.
Batch Percentile	$((\text{students scoring below this student}) / (\text{total students} - 1)) \times 100$, rounded to 1 decimal.	How this candidate compares to peers.
Assignment Avg Score	Mean of mark_awarded across all Completed assignments.	Code quality indicator.
Quiz Avg Score	Mean of quiz scores across all quizzes where student participated.	Knowledge retention.
Quiz Participation Rate	$(\text{quizzes participated} / \text{total quizzes scheduled}) \times 100$.	Engagement with knowledge checks.

14.2 Punctuality & Reliability Metrics

Metric	Formula / Definition	Purpose for Recruiter
Attendance Rate	$(\text{lectures where both halves attended} / \text{total lectures}) \times 100$. Expressed as %.	Reliability and discipline.
Full Attendance Rate	$(\text{lectures with 'both' halves} / \text{total lectures}) \times 100$.	Commitment level.
On-Time Submission Rate	$(\text{assignments submitted before deadline} / \text{total published assignments}) \times 100$.	Time management.
Punctuality Index	Weighted score: Full attendance = 1.0 per lecture, half = 0.4, absent = 0. Mean across all lectures $\times 100$.	Composite punctuality score (0–100).
Zero-Miss Streaks	Longest consecutive run of lectures with full attendance. Count stored as integer.	Sustained discipline.
Submission Lead Time (avg)	Mean of $(\text{deadline} - \text{submitted_at})$ in hours, for on-time submissions only.	How proactively they submit (vs last-minute).

14.3 Work Quality Metrics

Metric	Formula / Definition	Purpose for Recruiter
Code Quality Score (avg)	Mean of review_score across all Completed reviews (including late, since review still runs).	Technical proficiency.
Code Improvement Rate	Linear regression slope of review_score over time (assignment sequence). Positive = improving.	Learning trajectory.
Perfect Assignment Count	Count of assignments where review_score = 10.	Excellence incidents.
Below-Average Assignment Rate	(assignments with review_score < batch mean review_score / total assignments) × 100.	Risk indicator for recruiter.

14.4 Consistency & Growth Metrics

Metric	Formula / Definition	Purpose for Recruiter
Consistency Score	Standard deviation of (attendance marks + quiz marks + assignment marks) per lecture event. Lower SD = more consistent. Score = max(0, 100 – (SD × 10)), rounded.	Predictability of performance.
Growth Rate	Difference between the mean score of the last 30% of lectures vs first 30% of lectures. Positive = growth.	Trajectory over the internship.
Engagement Score	Composite: (quiz participation rate × 0.3) + (on-time submission rate × 0.4) + (attendance rate × 0.3). Expressed as 0–100.	Overall engagement with the program.
Topics Mastered	Count of topics where the associated assignment score ≥ 7 out of 10.	Depth of skill in the curriculum.
Curriculum Completion %	(topics with at least one completed lecture / total topics) × 100.	Program coverage.

14.5 Soft Skill Proxy Metrics

These metrics are inferred from system data and serve as proxies for soft skills that recruiters value.

Metric	Proxy For	How Computed
Punctuality Index (see 14.2)	Discipline, reliability	Computed from attendance data.
Submission Lead Time	Proactiveness, planning	Time between submission and deadline.
On-Time Submission Rate	Time management	Computed from submission timestamps.
Engagement Score	Intrinsic motivation	Composite of participation signals.

Code Improvement Rate	Learning agility, coachability	Regression slope of scores over time.
Consistency Score	Dependability, work ethic	SD of per-event scores.
Zero-Miss Streaks	Commitment, grit	Longest full-attendance run.
Quiz Participation Rate	Curiosity, self-assessment	Quiz CSV data.

Note to Developer: All metrics are computed at query time from the mark_ledger and submission collections. They are not stored as fields. Caching is recommended for the recruiter page (e.g., cache metrics per batch, invalidate on any new event). Cache TTL: 5 minutes.

15 Notification System

15.1 Notification Channels

- In-app notifications: bell icon with unread count. Notifications persist until manually dismissed.
- Email notifications: sent via configured email provider (SMTP or transactional email API; developer to configure). All emails include IMS branding and an unsubscribe link for non-critical notifications.

15.2 Notification Events

Event	Recipients	Channel	Timing
Lecture scheduled	All students in batch	In-app + Email	Immediately on creation.
Lecture reminder	All students in batch	In-app + Email	T-24h before start.
Lecture reminder	All students in batch	In-app + Email	T-1h before start.
Lecture cancelled	All students in batch	In-app + Email	Immediately on cancellation.
Assignment published	All students in batch	In-app + Email	Immediately when lecture marked Completed.
Assignment deadline approaching	Students who haven't submitted	In-app + Email	T-24h before deadline.
Assignment deadline passed	Students who didn't submit	In-app	Immediately after deadline.
Code review completed	Student (submission owner)	In-app + Email	Immediately on review completion.
Marks updated (override)	Affected student	In-app + Email	Immediately on admin override.
Attendance CSV uploaded	Teacher (confirmation)	In-app	Immediately on successful upload.
Quiz CSV rejected (error)	Teacher	In-app	Immediately on validation failure.
Code review error	Admin (all 4)	In-app + Email	Immediately on review API error.
Recalculation completed	Admin (all 4)	In-app	Immediately on completion.

16 Admin Override & Audit Trail

16.1 Override Types

Override Type	Fields Required	System Action
Student total mark adjustment	student_id, delta (positive or negative), reason (min 20 chars)	Creates mark_ledger entry with event_type='admin_override'. Recomputes total.
Specific event mark correction	student_id, event_id, new_mark, reason	Reverses original ledger entry. Creates corrected entry. Recomputes total.
Batch parameter change	batch_id, parameter_name, old_value, new_value, reason	Updates batch config. Optionally triggers full recalculation for batch.
Code review manual score entry	submission_id, manual_score (0–10), reason	Sets mark_awarded = manual_score. Closes Error status. Recomputes total.
Attendance CSV replacement	lecture_id, new_csv_file	Re-processes attendance. Recalculation triggered. Old CSV archived.

16.2 Audit Log Schema

Every override creates an immutable audit log entry. No audit entries can be deleted or edited, even by Admin.

Field	Type	Notes
audit_id	UUID	Auto-generated.
timestamp	datetime	Server time of action.
admin_id	ref	Which of the 4 admin accounts performed the action.
action_type	enum	Values: MARK_OVERRIDE, EVENT_CORRECTION, PARAM_CHANGE, MANUAL_SCORE, CSV_REPLACE, RECALC_TRIGGER.
target_type	enum	Values: STUDENT, LECTURE, BATCH, SUBMISSION.
target_id	UUID	ID of the affected entity.
old_value	JSON	Snapshot of state before change.
new_value	JSON	Snapshot of state after change.
reason	string	Admin-entered reason (min 20 chars, enforced server-side).

16.3 Audit Trail UI

- Admin sees a paginated audit trail table, filterable by: date range, admin_id, action_type, target_id.
- Each row is expandable to show old_value and new_value JSON.
- Admin can export the full audit trail (or filtered subset) as a CSV.

17 Database Schema (MongoDB Collections)

17.1 Collections Overview

Collection	Primary Key	Description
users	_id (UUID)	All users: admin, teacher, student. Role field distinguishes type.
batches	_id (UUID)	Batch records including recruiter UUID and config parameters.
topics	_id (UUID)	Curriculum topics per batch.
lectures	_id (UUID)	Lecture records per batch, linked to topics.
assignments	_id (UUID)	Assignment records linked to lectures.
submissions	_id (UUID)	Student assignment submissions.
mark_ledger	_id (UUID)	Immutable append-only log of every mark event per student.
quiz_csvs	_id (UUID)	Raw CSV uploads for quizzes, archived.
attendance_csvs	_id (UUID)	Raw CSV uploads for attendance, archived.
notifications	_id (UUID)	In-app notification records per user.
audit_log	_id (UUID)	Immutable admin override audit entries.
batch_configs	_id (UUID)	Per-batch configurable mark parameters.

17.2 Key Indexes

- mark_ledger: compound index on (student_id, created_at) for fast timeline queries.
- mark_ledger: index on lecture_id for recalculation queries.
- submissions: compound index on (assignment_id, student_id) unique, to prevent duplicate submissions.
- batches: index on recruiter_uuid for O(1) recruiter link lookup.
- notifications: compound index on (user_id, read, created_at).
- audit_log: index on (target_id, timestamp) for entity-specific audit queries.

18 API Endpoints

All endpoints are RESTful JSON APIs. Base path: /api/v1. Authentication via JWT Bearer token (except recruiter endpoints). Role-based middleware enforces access.

18.1 Auth

Method	Path	Access	Description
POST	/auth/login	All	Email + password login. Returns JWT.
POST	/auth/logout	All	Invalidates token.
POST	/auth/refresh	All	Refreshes JWT using refresh token.

18.2 Admin

Method	Path	Description
GET	/admin/dashboard	System summary stats.
POST	/admin/teachers	Create teacher account.
PUT	/admin/teachers/:id	Edit teacher account.
PATCH	/admin/teachers/:id/status	Activate/deactivate teacher.
POST	/admin/students	Create student account.
POST	/admin/students/bulk	Bulk create students via CSV.
PATCH	/admin/students/:id/status	Activate/deactivate student.
POST	/admin/batches	Create batch.
PUT	/admin/batches/:id	Edit batch.
PATCH	/admin/batches/:id/close	Close batch.
POST	/admin/batches/:id/recruiter-link	Generate recruiter link.
DELETE	/admin/batches/:id/recruiter-link	Revoke recruiter link.
POST	/admin/marks/override	Apply mark override with reason.
POST	/admin/marks/recalculate/:studentId	Trigger full recalculation for student.
POST	/admin/marks/recalculate-batch/:id	Trigger full recalculation for batch.
PUT	/admin/batch-config/:batchId	Update mark parameters for batch.
GET	/admin/audit-log	Paginated audit trail. Query params: from, to, adminId, actionType.
GET	/admin/audit-log/export	CSV export of audit trail.

18.3 Teacher

Method	Path	Description
GET	/teacher/dashboard	Upcoming lectures, pending uploads.
PUT	/teacher/profile	Edit own profile.
GET	/teacher/batches	List assigned batches.
POST	/teacher/topics	Create topic.
PUT	/teacher/topics/:id	Edit topic.
DELETE	/teacher/topics/:id	Delete topic (if no linked lecture).
POST	/teacher/topics/:id/notes	Upload notes file.
DELETE	/teacher/topics/:id/notes/:fileId	Delete notes file.
POST	/teacher/lectures	Create lecture.
PUT	/teacher/lectures/:id	Edit lecture (Scheduled status only).
PATCH	/teacher/lectures/:id/status	Transition lecture status.
POST	/teacher/lectures/:id/attendance	Upload attendance CSV.
POST	/teacher/lectures/:id/quiz	Upload quiz CSV.
GET	/teacher/students/:batchId	Student mark breakdown for batch.

18.4 Student

Method	Path	Description
GET	/student/dashboard	Total marks, events, upcoming lectures.
GET	/student/marks/history	Full mark ledger for own account.
GET	/student/assignments	List assignments (published + closed).
POST	/student/assignments/:id/submit	Submit GitHub URL.
GET	/student/assignments/:id/review	View code review result.
GET	/student/portfolio	Own portfolio (all metrics).
GET	/student/portfolio/pdf	Server-side PDF export of portfolio.
PUT	/student/profile	Edit own profile photo and name.

18.5 Recruiter (Public, No Auth)

Method	Path	Description
GET	/recruiter/:batchUuid	Batch overview page data.

GET	/recruiter/:batchUuid/students	List of all student portfolio summaries.
GET	/recruiter/:batchUuid/students/:id	Individual student full portfolio.

19 Non-Functional Requirements

19.1 Performance

- All dashboard API responses must return within 500 ms at p95 under 100 concurrent users.
- Recruiter batch page must load within 2 seconds at p95 (with metric caching, TTL 5 min).
- CSV processing (attendance/quiz) must complete within 10 seconds for batches of up to 500 students.
- Code review job queue must begin processing within 30 seconds of submission.

19.2 Security

- Passwords stored as bcrypt hashes (min 12 rounds).
- JWT expiry: access token 15 min, refresh token 7 days.
- Recruiter UUID is UUIDv4; minimum entropy of 122 bits. Not sequentially guessable.
- All API endpoints (except /recruiter/*) require valid JWT. Role middleware rejects unauthorised role access with HTTP 403.
- CSV uploads scanned for malicious content (file type validation, MIME check). Only CSV MIME type accepted.
- GitHub URL input sanitised and validated server-side before passing to code review service.
- Rate limiting: 60 requests/min per IP on auth endpoints; 200 requests/min per IP on all others.
- All data in transit encrypted via HTTPS (TLS 1.2+).

19.3 Reliability

- mark_ledger writes use MongoDB transactions to ensure atomicity. Partial writes roll back.
- Code review job queue persists jobs to MongoDB (Bull with MongoDB backend). Jobs survive server restart.
- Failed code review jobs are retried 3 times with exponential backoff (1 min, 5 min, 15 min) before triggering admin alert.
- Audit log is append-only; no update or delete operations are permitted at the database level (enforced via MongoDB collection-level rules).

19.4 Usability

- All forms display inline validation errors before submission.
- CSV upload screens show a template download link with the exact required format.
- All destructive actions (delete topic, revoke recruiter link, close batch) require a confirmation dialog.
- Dashboard data refreshes every 60 seconds without full page reload (polling or WebSocket).
- Application is responsive for desktop (1280px+) and tablet (768px+). Mobile is not a supported breakpoint.

19.5 Scalability

- Application stateless at server level; horizontal scaling via load balancer is supported.
- MongoDB Atlas or equivalent managed cluster recommended for production.
- Job queue (Bull/Redis) runs as a separate worker process, independently scalable.

20 Constraints & Assumptions

20.1 Constraints

- Tech stack is fixed: MongoDB, Express.js, React.js, Node.js. No substitutions.
- Code review service is an existing external system. IMS treats it as a black box API. Integration is limited to: send GitHub URL, receive { score: number, feedback: string }.
- Attendance CSVs are generated by an existing third-party browser extension. IMS does not control or modify that extension's output format. The teacher is responsible for ensuring the CSV matches the required IMS format before upload.
- Quiz platforms (Kahoot, Google Forms) do not have a direct API integration. Teacher manually exports a CSV from those platforms and uploads to IMS. IMS does not connect to Kahoot or Google APIs.
- Google Meet scheduling is external. IMS stores the Meet URL as a plain string; it does not create or manage Meet sessions.
- Email delivery depends on the configured email provider. IMS is not responsible for delivery failures caused by the email provider.

20.2 Assumptions

- Each student is enrolled in exactly one batch at a time.
- Each lecture is associated with exactly one batch.
- A teacher account is created before any curriculum or lecture can be created for a batch.
- The code review API is available and returns a valid JSON response with 'score' (0–10) and 'feedback' fields. IMS does not define the scoring criteria; it accepts whatever score the service returns.
- The third-party attendance extension exports a CSV with at least the columns student_email and a half-attendance indicator. The teacher maps those to the IMS-required format (student_email, half_attended) before upload.
- Admin accounts are seeded at deployment. There is no self-registration or admin creation via the UI.
- The system is single-tenant (one organisation). Multi-tenancy is out of scope.

21 Glossary

Term	Definition
IMS	Intern Management System — the application described in this document.
MERN	MongoDB, Express.js, React.js, Node.js.
JWT	JSON Web Token — used for stateless authentication.
CSV	Comma-Separated Values — file format used for attendance and quiz uploads.
Ledger	The mark_ledger collection: an append-only record of every mark event.
Recalculation	The process of recomputing a student's total marks from scratch using all ledger entries.
Recruiter Link	A publicly accessible, read-only URL that displays a batch's student portfolios.
Batch UUID	The UUIDv4 used in the recruiter URL; distinct from the internal batch _id.
Code Review	Automated analysis of a GitHub repository; returns a score (0–10) and feedback text.
Pending Mark	A mark entry held in suspension due to a code review error; resolved by admin.
Half-Attendance	A student who attended only one of the two halves of a lecture.
Excess Marks	Marks that would push a student above 100; logged but not added to the total.
Audit Trail	The immutable log of all admin overrides and system recalculations.
Batch Config	The set of configurable mark parameters for a batch, editable by Admin.
Delta	The difference in marks between an old and new calculation, applied to the running total.
Engagement Score	A composite metric (0–100) combining quiz participation, submission timeliness, and attendance.
Consistency Score	A metric derived from the standard deviation of per-event scores; lower SD = higher consistency.
Growth Rate	The difference in mean score between the last 30% and first 30% of lectures; positive = improvement.

— End of Document —